

Consigne de remise pour le cours de projet de développement SGBD

Auteur : Nicolas Wattiaux

Date de remise : 31 Mai 2026

Format de remise : - Lien GitHub envoyé par email et Zip sur Moodle

1. Contexte

Ce projet vise à évaluer votre maîtrise de l'ensemble des concepts vus en cours, en deux volets :

1. **Modélisation relationnelle** : conception d'un schéma de base de données solide, avec relations 1:N, tables de jonction N:M, clés étrangères et agrégations — notions vues en SQL et à appliquer ici via **Prisma**.
2. **Développement applicatif** : réalisation d'une application de bureau avec **Electron**, en respectant une architecture moderne (main / preload / renderer) et en utilisant Prisma comme ORM pour l'accès aux données. La partie renderer sera évaluée globalement, mais la bonne application des concepts vus dans le cours parallèle (Angular) sera vérifiée. Une liste du minimum requis est disponible en annexe de ce document.

Vous devez réaliser une application fonctionnelle, inspirée du repos de référence : [todos-app-electron](#).

2. Exigences Techniques

2.1. Architecture Globale

- **Framework principal** : Electron.
 - **Backend** : Node.js avec **Prisma** pour l'interaction avec la base de données.
 - **Frontend (Renderer)** : Angular
 - **Base de données** : Utilisation de **SQLite** via Prisma (provider [sqlite](#), fichier local). Aucune autre base de données n'est acceptée.
 - **Structure** : Respecter l'architecture vue en cours (séparation claire entre main, preload et renderer).
-

2.2. Fonctionnalités Obligatoires

- **Fonctionnalités de base** : L'application doit permettre de créer, lire, mettre à jour et supprimer des données (CRUD) dans chaque table.
- **Persistance des données** : Utilisation de Prisma pour interagir avec la base de données.
- **Interface utilisateur** : Fluide et intuitive.
- **Scripts npm** : Le projet doit se lancer avec la commande [npm run start](#). Si des étapes supplémentaires sont nécessaires, elles doivent être détaillées dans le README.

2.3. Contraintes Techniques — Qualité du code

Critère	Ce qu'on attend
Pas de dépendances cloud	Toutes les données sont stockées en SQLite local
TypeScript strict	
Modularité	Un composant = une responsabilité ; la logique métier est dans les services
Nommage	Noms explicites (pas <code>data</code> , <code>temp</code> , <code>x</code>)
Gestion des erreurs	Les appels IPC/Prisma ont un <code>try/catch</code>
Documentation	Un README clair avec les instructions d'installation et de lancement

2.4. Exigences Angular

Votre renderer Angular doit démontrer la maîtrise des concepts vus en cours : composants standalone, signals, computed, @for/@if, services avec DI, input/output, formulaires réactifs et routage.

Le détail des notions attendues est disponible en annexe (slide 5.1).

2.5. Exigences de modélisation — Prisma

Vous n'écrivez pas de SQL brut — Prisma le génère — mais la **structure de votre schéma** et l'**utilisation du Prisma Client** doivent refléter les notions vues en cours : relations 1:N, tables de jonction N:M, ON DELETE, JOIN/include et agrégats.

Le détail des notions attendues est disponible en annexe (slide 5.2).

3. Livrables

3.1. Code Source

- **Dépôt GitHub** : Le projet doit être hébergé sur GitHub.
 - **Structure du dépôt** :
 - Un fichier `README.md`.
 - Un fichier `.gitignore` adapté.
 - Une configuration Prisma fonctionnelle (`prisma/schema.prisma`).
 - Un schéma de la base de données au format **draw.io** (ou autre)
 - Un script permettant de peupler la base de données avec des données de test si nécessaire.
-

3.2. Remise

- **Format** : Un lien GitHub envoyé par email le **31 Mai 2026**.
 - **Dépôt du zip** : Le projet doit également être déposé en version zippée sur la plateforme **Moodle** avant la même échéance.
 - **Critère de recevabilité** : Le projet doit se lancer avec `npm run start` ou en suivant les consignes du README. Si le projet ne se lance pas, il sera noté 0.
-

4. Critères d'évaluation

Critère	Pondération
Fonctionnalité	10%
Architecture et qualité du code	10%
Modélisation (schéma DB, relations, seed)	10%
Compréhension du code (oral)	70%

Bonus : Tests unitaires, gestion avancée des erreurs, ou fonctionnalités supplémentaires pertinentes.

Ces notions seront **vérifiées à l'oral** : on vous demandera d'expliquer vos choix de modélisation et de faire le lien avec le SQL équivalent.

5. Annexes — Récapitulatifs des exigences

5.1. Notions Angular à démontrer

Notion	Ce qu'on attend	Syntaxe clé	Priorité
Composant	Au moins 3 composants standalone distincts	<code>@Component({ standalone: true, ... })</code>	Obligatoire
TypeScript	Typier les données avec des interfaces	<code>interface Todo { id: number; titre: string }</code>	Obligatoire
Signals	Utiliser <code>signal()</code> pour l'état local réactif	<code>count = signal(0) / count.set(1) / count.update(v => v+1)</code>	Obligatoire
Computed	Au moins un signal dérivé	<code>total = computed(() => this.items().length)</code>	Obligatoire
@for / @if	Afficher une liste, conditionner l'affichage	<code>@for (item of items()); track item.id) / @if (condition)</code>	Obligatoire
Service + DI	Un service injecté dans un composant	<code>@Injectable({ providedIn: 'root' }) + inject(MonService)</code>	Obligatoire
Singleton	Service partagé via <code>providedIn: 'root'</code>	Une seule instance pour toute l'appli	Obligatoire
input()	Passer des données du parent vers l'enfant	<code>message = input.required<string>()</code>	Obligatoire
output()	Émettre un événement de l'enfant vers le parent	<code>supprime = output<number>() + supprime.emit(id)</code>	Obligatoire
Formulaires	Au moins un formulaire avec <code>ReactiveFormsModule</code>	<code>fb.group({ titre: ['', Validators.required] })</code>	Obligatoire
Routage	Au moins 2 routes + <code><router-outlet></code>	<code>app.routes.ts + provideRouter(routes)</code>	Obligatoire
RouterLink	Navigation via liens	<code>routerLink="/ma-page"</code>	Obligatoire
effect()	Réagir à un changement de signal (side effect)	<code>effect(() => console.log(this.count()))</code>	Bonus

Ces notions seront **vérifiées à l'oral** : on vous demandera d'expliquer vos choix et de faire le lien avec les concepts vus en cours.

5.2. Notions SQL appliquées via Prisma

Notion	Ce qu'on attend	Implémentation Prisma	Priorité
Tables	Au minimum 7 modèles distincts	7 blocs <code>model</code> dans <code>schema.prisma</code>	Obligatoire
Clé primaire	Chaque table a un identifiant unique	<code>id Int @id @default(autoincrement())</code>	Obligatoire
Relation 1:N	Au moins 1 relation un-à-plusieurs avec clé étrangère	<code>@relation</code> + champ <code>foreignKey</code> + <code>onDelete</code>	Obligatoire
Table de jonction N:M	Au moins 1 relation plusieurs-à-plusieurs	Table pivot explicite avec <code>@@id([aId, bId])</code>	Obligatoire
ON DELETE	Comportement défini sur suppression	<code>onDelete: Cascade</code> ou <code>Restrict</code>	Obligatoire
JOIN / include	Données liées récupérées ensemble	<code>prisma.post.findMany({ include: { auteur: true } })</code>	Obligatoire
Agrégat / comptage	Au moins une stat ou comptage visible dans l'UI	<code>prisma.todo.count()</code> ou <code>groupBy + count</code>	Obligatoire
CRUD complet	Créer, lire, modifier, supprimer sur au moins 1 entité	<code>create, findMany, update, delete</code>	Obligatoire
Champs optionnels	Utilisation de champs nullable	<code>description String?</code>	Recommandé
Enum Prisma	Au moins un champ à valeur fixe	<code>enum Statut { ACTIF INACTIF }</code>	Bonus

5.3. Architecture Electron

Notion	Attendu	Détail
Main process	<code>main.ts</code> correct	Crée la <code>BrowserWindow</code> , charge l'app
Preload	<code>contextBridge</code>	<code>ipcRenderer.invoke</code> exposé au renderer
Renderer	App Angular	Via l'API preload uniquement
IPC	Main ↔ renderer	<code>ipcMain.handle</code> + <code>ipcRenderer.invoke</code>
npm run start	Une seule commande	Détailler dans le README si nécessaire
SQLite local	Pas de cloud	<code>provider = "sqlite"</code> dans <code>schema.prisma</code>